

Progress Report

Day 16 — Protected Routing and Shared Application Layout

Task Management MERN Stack Platform

Shafin Nigamana | 04 June 2026

1. Day-16 Objective

The objective of Day 16 was to secure application routes using authentication-based access control, establish a reusable application layout, and display authenticated user information throughout the application interface.

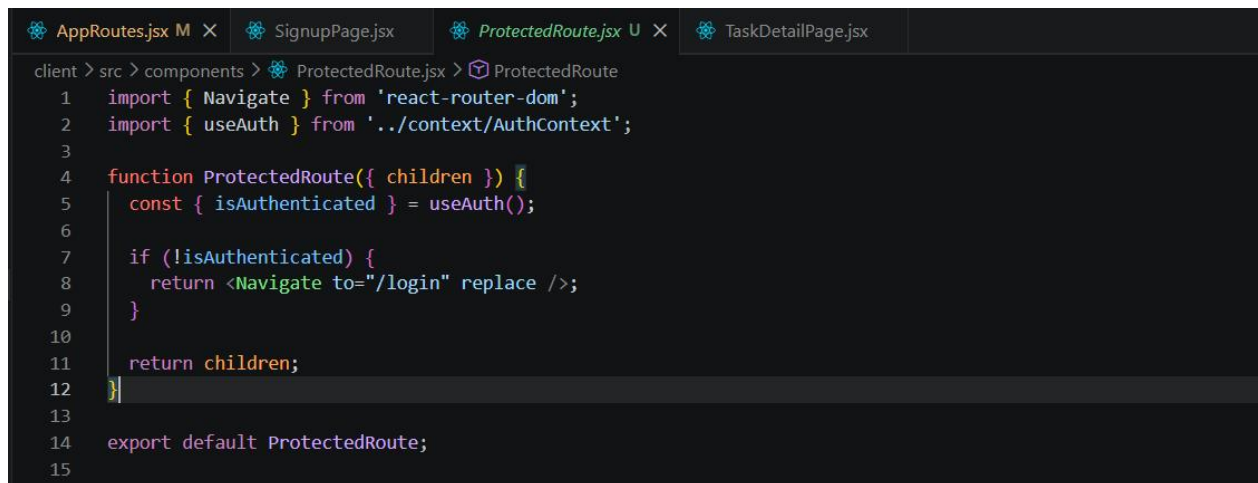
2. Tasks Completed

Protected Route Implementation

- Developed a reusable ProtectedRoute component to secure application pages from unauthorized access.

Implemented functionality:

- Authentication verification using AuthContext
 - Automatic redirection to the login page for unauthenticated users
 - Protected access to authenticated application modules
 - Route guarding using React Router
- Ensured users cannot directly access protected pages without valid authentication.



```
client > src > components > ProtectedRoute.jsx > ProtectedRoute
1  import { Navigate } from 'react-router-dom';
2  import { useAuth } from '../context/AuthContext';
3
4  function ProtectedRoute({ children }) {
5    const { isAuthenticated } = useAuth();
6
7    if (!isAuthenticated) {
8      return <Navigate to="/login" replace />;
9    }
10
11    return children;
12  }
13
14  export default ProtectedRoute;
15
```

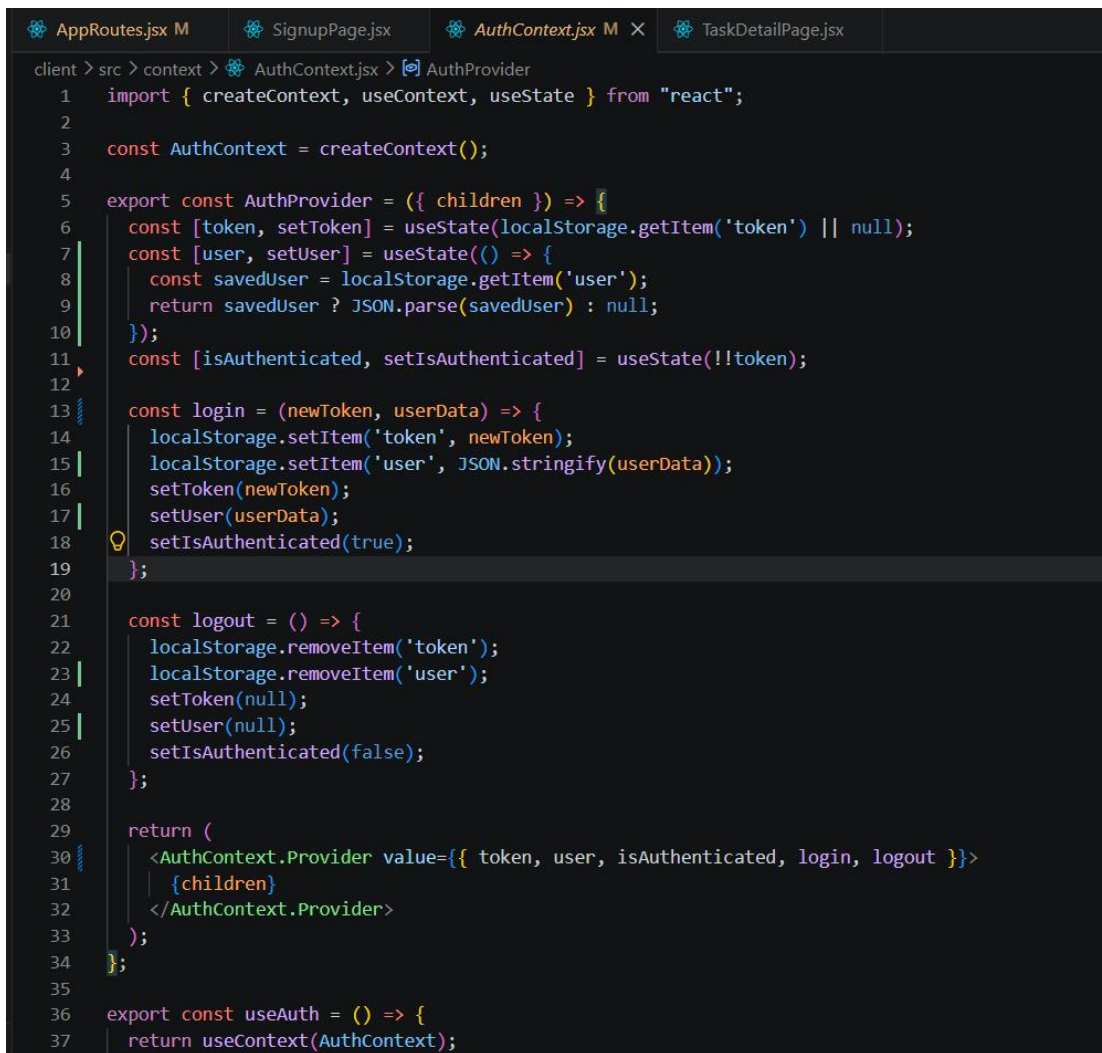
Figure 2.1 — Protected route implementation for authentication enforcement.

Authentication Context Enhancement

- Extended the authentication context to support user profile persistence.

Implemented features:

- User information storage
 - User information restoration after page refresh
 - Authentication state persistence
 - Logout state cleanup
- Configured local storage persistence for both JWT tokens and authenticated user information.



```
client > src > context > AuthContext.jsx > AuthProvider
1 import { createContext, useContext, useState } from "react";
2
3 const AuthContext = createContext();
4
5 export const AuthProvider = ({ children }) => {
6   const [token, setToken] = useState(localStorage.getItem('token') || null);
7   const [user, setUser] = useState(() => {
8     const savedUser = localStorage.getItem('user');
9     return savedUser ? JSON.parse(savedUser) : null;
10  });
11   const [isAuthenticated, setIsAuthenticated] = useState(!!token);
12
13   const login = (newToken, userData) => {
14     localStorage.setItem('token', newToken);
15     localStorage.setItem('user', JSON.stringify(userData));
16     setToken(newToken);
17     setUser(userData);
18     setIsAuthenticated(true);
19   };
20
21   const logout = () => {
22     localStorage.removeItem('token');
23     localStorage.removeItem('user');
24     setToken(null);
25     setUser(null);
26     setIsAuthenticated(false);
27   };
28
29   return (
30     <AuthContext.Provider value={{ token, user, isAuthenticated, login, logout }}>
31       {children}
32     </AuthContext.Provider>
33   );
34 };
35
36 export const useAuth = () => {
37   return useContext(AuthContext);
38 }
```

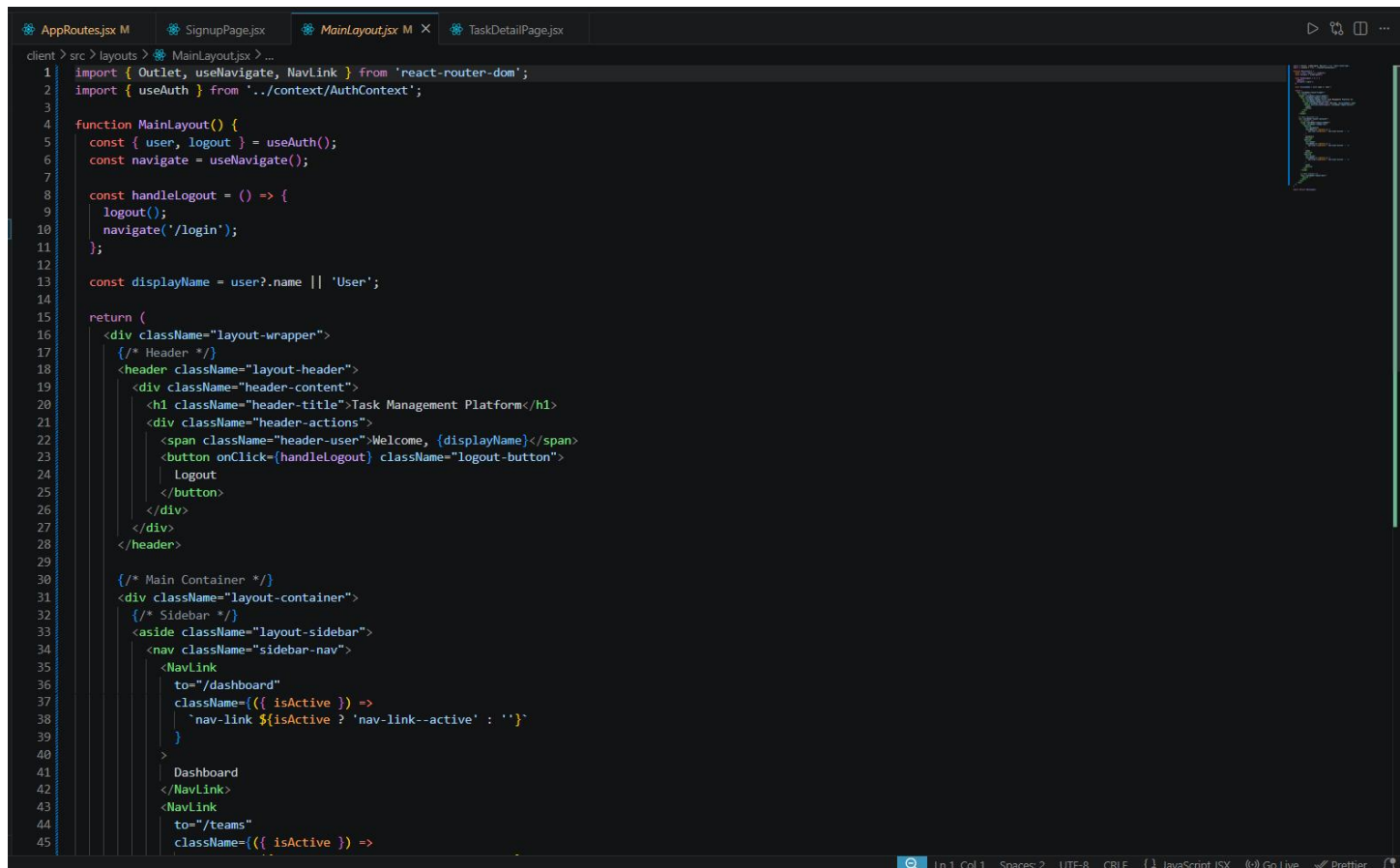
Figure 2.2 — Authentication context enhanced with user state persistence.

Shared Layout Development

- Implemented a centralized application layout to provide a consistent user experience across protected pages.

Layout components:

- Header section
 - Navigation sidebar
 - Dynamic content area
- Integrated React Router Outlet to render page-specific content within the shared layout structure.



```
1 import { Outlet, useNavigate, NavLink } from 'react-router-dom';
2 import { useAuth } from '../context/AuthContext';
3
4 function MainLayout() {
5   const { user, logout } = useAuth();
6   const navigate = useNavigate();
7
8   const handleLogout = () => {
9     logout();
10    navigate('/login');
11  };
12
13  const displayName = user?.name || 'User';
14
15  return (
16    <div className="layout-wrapper">
17      <div className="layout-header">
18        <div className="header-content">
19          <h1 className="header-title">Task Management Platform</h1>
20          <div className="header-actions">
21            <span className="header-user">Welcome, {displayName}</span>
22            <button onClick={handleLogout} className="logout-button">
23              Logout
24            </button>
25          </div>
26        </div>
27      </div>
28
29      <div className="layout-container">
30        <div className="layout-sidebar">
31          <div className="sidebar-nav">
32            <NavLink
33              to="/dashboard"
34              className={({ isActive }) =>
35                `nav-link ${isActive ? 'nav-link--active' : ''}`
36            >
37              Dashboard
38            </NavLink>
39            <NavLink
40              to="/teams"
41              className={({ isActive }) =>
```

Figure 2.3 — Shared application layout implementation.

Route Structure Refactoring

- Refactored application routing to support nested protected routes.

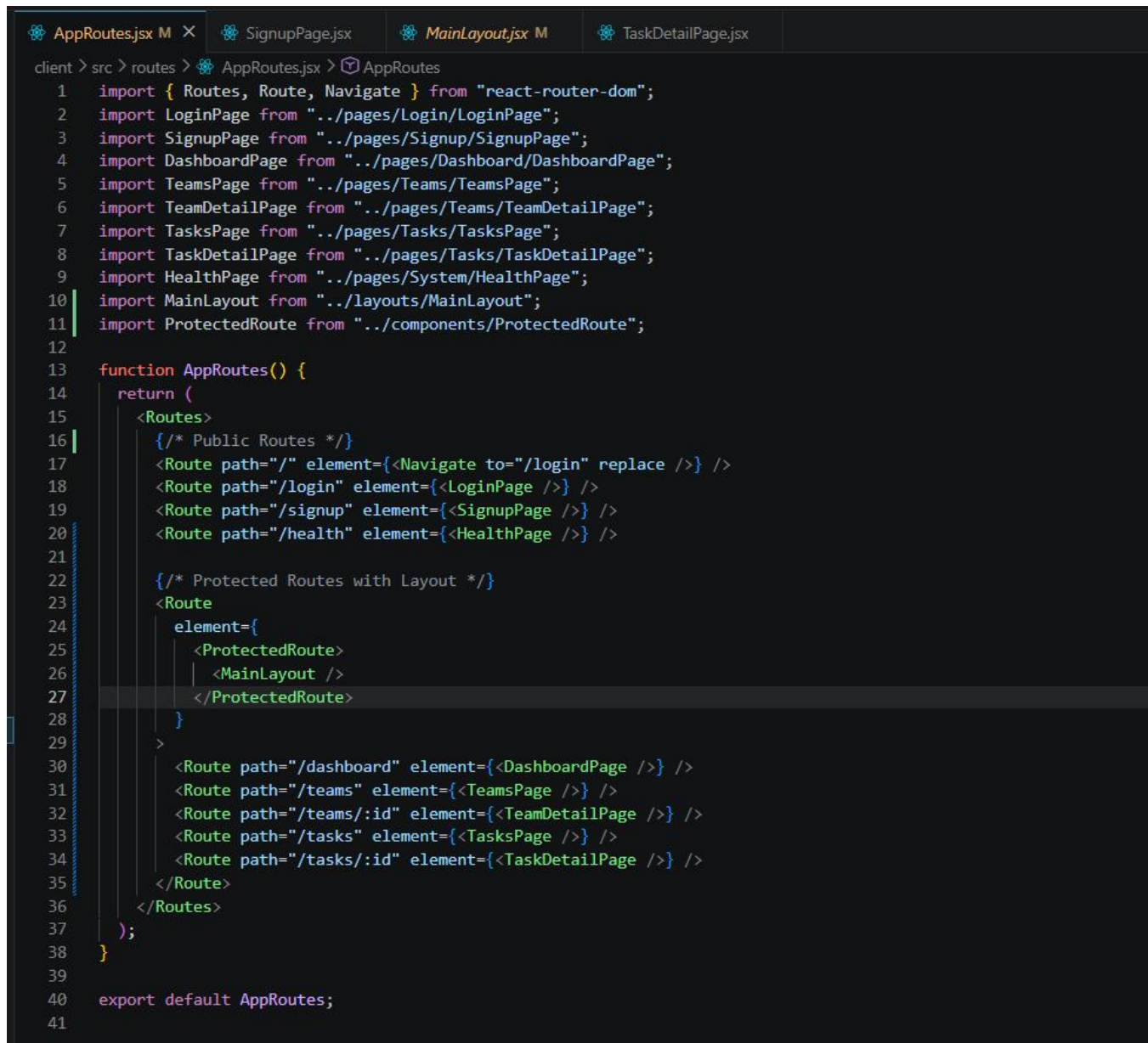
Protected routes configured:

- Dashboard

- Teams
- Team Details
- Tasks
- Task Details

Public routes retained:

- Login
 - Signup
 - Health Monitoring
- Improved route organization and maintainability using nested routing patterns.



```
client > src > routes > AppRoutes.jsx > AppRoutes
1  import { Routes, Route, Navigate } from "react-router-dom";
2  import LoginPage from "../pages/Login/LoginPage";
3  import SignupPage from "../pages/Signup/SignupPage";
4  import DashboardPage from "../pages/Dashboard/DashboardPage";
5  import TeamsPage from "../pages/Teams/TeamsPage";
6  import TeamDetailPage from "../pages/Teams/TeamDetailPage";
7  import TasksPage from "../pages/Tasks/TasksPage";
8  import TaskDetailPage from "../pages/Tasks/TaskDetailPage";
9  import HealthPage from "../pages/System/HealthPage";
10 import MainLayout from "../layouts/MainLayout";
11 import ProtectedRoute from "../components/ProtectedRoute";
12
13 function AppRoutes() {
14   return (
15     <Routes>
16       { /* Public Routes */ }
17       <Route path="/" element={<Navigate to="/login" replace />} />
18       <Route path="/login" element={<LoginPage />} />
19       <Route path="/signup" element={<SignupPage />} />
20       <Route path="/health" element={<HealthPage />} />
21
22       { /* Protected Routes with Layout */ }
23       <Route
24         element={
25           <ProtectedRoute>
26             <MainLayout />
27           </ProtectedRoute>
28         }
29       >
30         <Route path="/dashboard" element={<DashboardPage />} />
31         <Route path="/teams" element={<TeamsPage />} />
32         <Route path="/teams/:id" element={<TeamDetailPage />} />
33         <Route path="/tasks" element={<TasksPage />} />
34         <Route path="/tasks/:id" element={<TaskDetailPage />} />
35       </Route>
36     </Routes>
37   );
38 }
39
40 export default AppRoutes;
41
```

Figure 2.4 — Protected routing configuration using nested routes.

Dashboard Layout Integration

- Integrated the Dashboard page into the newly developed shared layout.

Verified functionality:

- Header visibility
- Sidebar navigation rendering
- Content area rendering through Outlet
- Layout consistency across authenticated pages

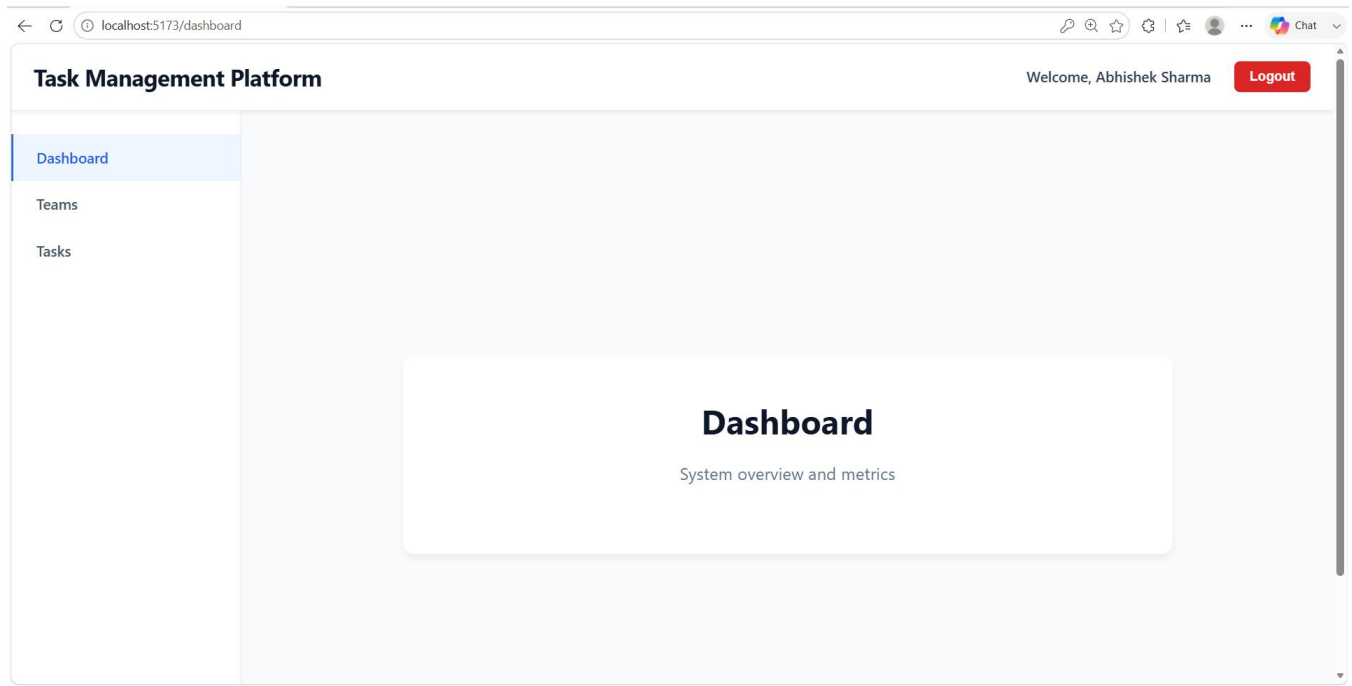


Figure 2.5 — Authenticated dashboard rendered within shared layout.

User Information Display

- Implemented authenticated user information display within the application header.

Displayed information:

- Logged-in user name
 - Authentication status
 - Logout action
- Connected header content to AuthContext for dynamic user rendering.

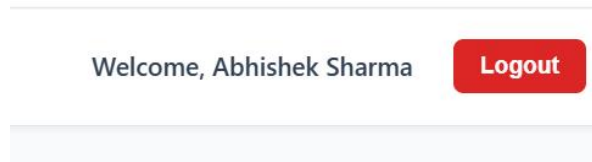


Figure 2.6 — Logged-in user information displayed in application header.

Authentication Flow Verification

- Verified complete route protection behavior throughout the application.

Validated scenarios:

- Authenticated user access
- Unauthenticated route access attempts
- Automatic redirection to login page
- Logout and session cleanup behavior

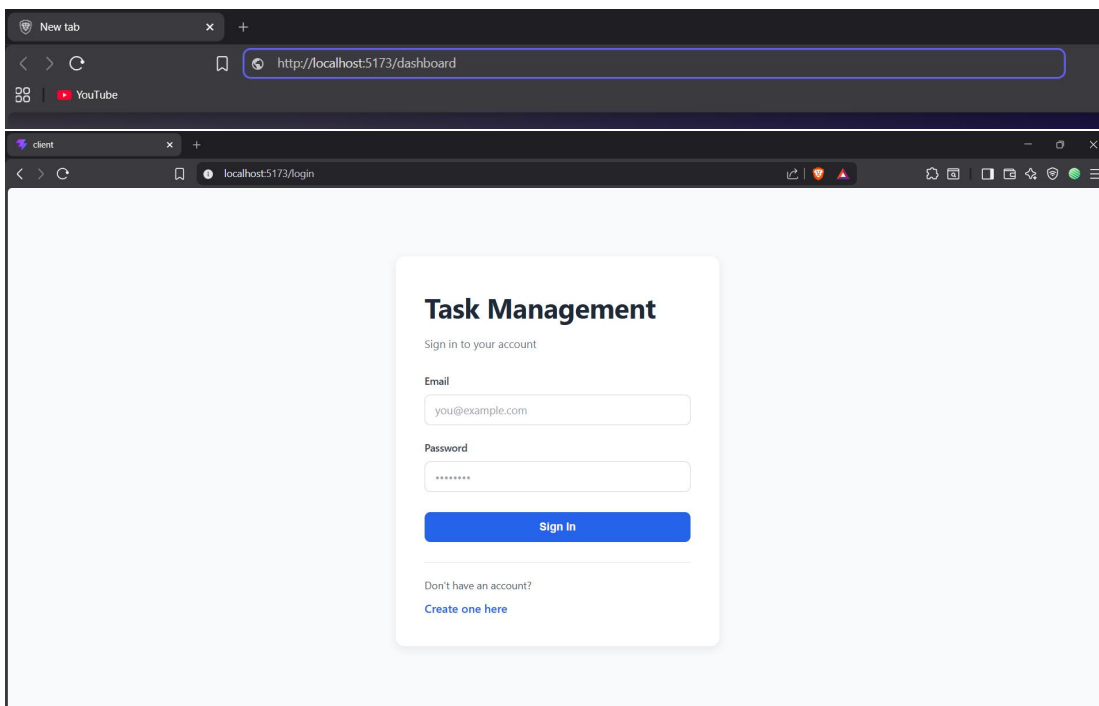


Figure 2.7 — Automatic redirection of unauthenticated users to the login page.

3. Conclusion

Day 16 focused on strengthening application security and establishing a consistent authenticated user experience. Protected routing was successfully implemented, a reusable application layout was developed, and user information was integrated into the interface through centralized authentication state management. The application is now prepared for dashboard enhancements and protected feature development in subsequent phases.